Data Structure and algorithms (LAB)

Lab Task

Name: UzairAlam

TASK 1: INSERTION SORT

Loop Condtions:

Outer loop: (int i = 1; i < size; i++)

Inner loop: (j >= 0 && arr[j] > temp) if condition true swapping occure…

In the last arr[j+1] = key; to fill the remaining value…

Given Array:

int marks[5] = {45, 20, 35, 10, 30};

Pass 1: Key = 20

20 is smaller than 45, so shift 45 to the right and insert 20.

Array: {20, 45, 35, 10, 30}

Pass 2: Key = 35

35 is smaller than 45, so shift 45 to the right and insert 35.

Array: {20, 35, 45, 10, 30}

Pass 3: Key = 10

Shift 45, 35, and 20 to the right, then insert 10 at the beginning.

Array: {10, 20, 35, 45, 30}

Pass 4: Key = 30

Shift 45 and 35 to the right, then insert 30 after 20.

Array: {10, 20, 30, 35, 45}

Final Sorted Array (Ascending Order):

{10, 20, 30, 35, 45}

TASK 2: BUBBLE SORT

Loop Condtions:

Outer loop: (int i = 0; i < size - 1; i++)

Inner loop: (int j = 0; j < size - 1 - i; j++)

Swapping condition: arr[j] > arr[j + 1] if this happen then swapping occure…

Given Array:

int waiting[5] = {40, 15, 30, 10, 25};

Pass 1:

- Compare 40 and 15 → Swap.
- Compare 40 and 30 → Swap.
- Compare 40 and 10 → Swap.
- Compare 40 and 25 → Swap.

Array after Pass 1: {15, 30, 10, 25, 40}

Pass 2:

- Compare 15 and 30 → No swap.
- Compare 30 and 10 → Swap.
- Compare 30 and 25 → Swap.

Array after Pass 2: {15, 10, 25, 30, 40}

Pass 3:

- Compare 15 and 10 → Swap.
- Compare 15 and 25 → No swap.

Array after Pass 3: {10, 15, 25, 30, 40}

Pass 4:

- Compare 10 and 15 → No swap.

Array after Pass 4: {10, 15, 25, 30, 40}

Total Number of Swaps: 9

Final Sorted Array:

{10, 15, 25, 30, 40}

TASK 3: SELECTION SORT

Loop Condtions:

Outer loop: (int i = 0; i < size - 1; i++)

Inner loop: (int j = 0; j < size - 1 - i; j++)

Selecting Index using this condition: arr[index] > arr[j]

In the last we again reached at outer loop, and then there we do swapping…

Given Array:

int scores[5] = {55, 25, 45, 10, 35};

Pass 1:

Unsorted array: {55, 25, 45, 10, 35}

Minimum element = 10

Swap 10 with 55.

Array after Pass 1: {10, 25, 45, 55, 35}

Pass 2:

Unsorted array: {25, 45, 55, 35}

Minimum element = 25

No swap is needed because 25 is already in the correct position.

Array after Pass 2: {10, 25, 45, 55, 35}

Pass 3:

Unsorted array: {45, 55, 35}

Minimum element = 35

Swap 35 with 45.

Array after Pass 3: {10, 25, 35, 55, 45}

Pass 4:

Unsorted array: {55, 45}

Minimum element = 45

Swap 45 with 55.

Array after Pass 4: {10, 25, 35, 45, 55}

Final Sorted Array:

{10, 25, 35, 45, 55}

TASK 4: QUICK SORT

Ok, so in quick sort we use recursion… there are two function one we use for partition where we do the swaping and chose the pivot, and one for quick sort where we do recusrion and sort left and right subArray…

Given Array:

int prices[5] = {35, 15, 50, 10, 25};

First Partition:

Pivot = 25

Initial array: {35, 15, 50, 10, 25}

Step 1: Compare 35 with 25. No swap is needed.

Step 2: Compare 15 with 25. Swap 15 with 35.

Array: {15, 35, 50, 10, 25}

Step 3: Compare 50 with 25. No swap is needed.

Step 4: Compare 10 with 25. Swap 10 with 35.

Array: {15, 10, 50, 35, 25}

Step 5: Swap the pivot 25 with 50.

Array after the first partition: {15, 10, 25, 35, 50}

Final position of the first pivot: Index 2 (position 3).

Second Partition (Left Subarray):

Subarray: {15, 10}

Pivot = 10

Compare 15 with 10. No swap is needed.

Swap the pivot 10 with 15.

Array after the second partition: {10, 15, 25, 35, 50}

Third Partition (Right Subarray):

Subarray: {35, 50}

Pivot = 50

Compare 35 with 50. No swap is needed because 35 is smaller than the pivot.

Array after the third partition: {10, 15, 25, 35, 50}

Final Sorted Array:

{10, 15, 25, 35, 50}